Advanced Prompting technique - Chain of Thoughts

Task1: Prepare a test strategy document based on the domain, using a chain-of-thought approach
Example Taken : Domain: Stock Market (Trading/Financial)

Prompt :

Context : As a QA Lead with Trading/financial domain generate a test strategy document for Order Management
Instruction :
Ask me 5 question for Test strategy document creation
After providing the answers ask the next question
Emphasize all major section headers (eg ,Objective,Scope,Test Types, risks) using distinct formatting such as bold,heading styles ensuring the document is well structured.
Easy to Navigate and Presentation Ready
Constraints : Follow the Enterprise standard

Output
Great—let's build this step by step to ensure the **test strategy is enterprise-grade and tailored to your trading domain**.

Question 1 of 5

To shape the **Objective** and **Scope** sections effectively:

👉 What type of **Order Management System (OMS)** are we dealing with?

Please specify:

- Asset classes (e.g., Equities, Options, Futures, FX, Crypto)
- Order types supported (e.g., Market, Limit, Stop, Algo orders)
- Market coverage (e.g., US, EMEA, APAC exchanges)
- Whether it integrates with external systems (e.g., FIX gateways, clearing systems, market data feeds)

Once you answer, I'll proceed with the next question.

Equities

Great—Equities OMS noted. That helps anchor the **Scope** significantly.

Question 2 of 5

To define **Test Types**, **Environment Strategy**, and **Integration Coverage**:

👉 What are the **key system integrations and interfaces** involved in your OMS?

Please specify:

- Protocols (e.g., FIX, REST, MQ, Kafka)
- External systems (e.g., exchanges, brokers, clearing/settlement systems)
- Market data providers (if any)
- Any upstream/downstream systems (e.g., Risk systems, Portfolio Management, Compliance/Surveillance)

Once you provide this, I'll move to the next question.

brokers

Got it—broker integrations noted (likely via FIX, but we'll confirm details as we go).

Question 3 of 5

To properly define **Test Approach**, **Test Data Strategy**, and **Non-Functional Testing (Performance, Scalability)**:

👉 What is the **expected order volume and system load profile**?

Please include:

- Average orders per day
- Peak load (orders per second or burst scenarios)
- Trading hours coverage (e.g., market open spikes, after-hours trading)
- Any latency expectations (e.g., order acknowledgment time, execution turnaround SLAs)

This will help shape performance, stress, and reliability testing in the strategy.

Once you answer, I'll move to the next question.

150

Thanks—that gives a baseline (we'll treat **150 as average daily orders** unless you уточнить peak later).

Question 4 of 5

To define **Test Environments**, **Release Strategy**, and **Execution Governance**:

👉 What is your **delivery model and release cadence**?

Please specify:

- Development methodology (e.g., Agile, Scrum, SAFe, Waterfall)
- Release frequency (e.g., weekly, bi-weekly, monthly)
- Environments available (e.g., DEV, QA, UAT, PROD, DR)
- Any CI/CD tools in use (e.g., Jenkins, GitLab CI, Azure DevOps)

This will help structure **test cycles, entry/exit criteria, and automation strategy**.

Once you provide this, I'll move to the final question.

Agile,bi-weekly,DEV, QA, UAT, PROD,Azure DevOps

Perfect—this gives strong structure for execution planning.

Question 5 of 5

To finalize **Risk Management**, **Test Coverage Depth**, and **Compliance Considerations**:

👉 Are there any **regulatory, compliance, or critical risk areas** we must prioritize?

Please specify:

- Regulatory requirements (e.g., SEC, FINRA, MiFID II, audit trails)
- Key risk areas (e.g., order misrouting, duplicate orders, latency issues, trade breaks)
- Data sensitivity (e.g., PII, financial data protection)

- Any audit or reporting requirements

Once you answer this, I will generate your **complete, enterprise-standard, presentation-ready Test Strategy document** with clearly structured sections like **Objective, Scope, Test Types, Risks, Environments, Governance, etc.**

FINRA,latency issues,PII,Audit report

Excellent—thank you. Based on your inputs, here is a **presentation-ready, enterprise-standard Test Strategy Document** tailored for an **Equities Order Management System (OMS)**.

Test Strategy Document

TEST STRATEGY DOCUMENT – ORDER MANAGEMENT SYSTEM (OMS)

---

1. Objective

The objective of this Test Strategy is to define a **robust, scalable, and compliant testing approach** for the Equities OMS to ensure:

- Accurate and reliable **order lifecycle management**
- Seamless integration with **broker systems**
- Compliance with **regulatory standards (FINRA)**
- Adherence to **latency and performance expectations**
- Protection of **sensitive financial and PII data**

---

2. Scope

In Scope

- Equity order processing (Order Capture → Validation → Routing → Execution → Allocation)
- Broker integration (order routing and execution feedback)
- Order types (Market, Limit – assumed baseline)
- Trade lifecycle validation and state transitions
- Audit trail validation and reporting
- Data validation for financial and client-related information

Out of Scope

- Non-equity asset classes (Derivatives, FX, Crypto)
- Third-party broker internal system validation
- UI/UX enhancements not impacting core trading workflows (unless specified)

---

3. System Overview

The OMS facilitates:

- Order entry and validation
- Routing to brokers
- Execution tracking
- Trade confirmation and reporting

Key Integration:

- External Broker Systems (likely via FIX or API-based communication)

---

4. Test Approach

4.1 Testing Methodology

- Agile-based iterative testing aligned with **bi-weekly sprints**
- Continuous testing integrated within **Azure DevOps CI/CD pipelines**
- Shift-left approach with early QA involvement

4.2 Test Levels

- Unit Testing (by Dev team)
- Integration Testing (OMS ↔ Broker)
- System Testing (end-to-end workflows)
- User Acceptance Testing (UAT)

4.3 Test Design Techniques

- Boundary Value Analysis (order limits, quantities)
- Equivalence Partitioning
- State Transition Testing (order lifecycle states)
- Risk-Based Testing (focus on latency, compliance, audit)

---

5. Test Types

5.1 Functional Testing

- Order creation, modification, cancellation
- Validation of order attributes (price, quantity, symbol)
- Broker routing and acknowledgment handling
- Execution status updates and trade lifecycle

5.2 Integration Testing

- OMS ↔ Broker communication validation
- Message format validation (e.g., FIX messages if applicable)
- Error handling and retry mechanisms

5.3 Performance & Latency Testing

- Validate system behavior under:
  - Average load (~150 orders/day)
  - Peak/burst scenarios (simulated)
- Measure:
  - Order acknowledgment time
  - Execution turnaround time
- Identify latency bottlenecks

5.4 Security Testing

- PII data protection validation
- Authentication & authorization checks
- Data encryption validation (in transit & at rest)

5.5 Compliance Testing

- FINRA regulatory requirements validation
- Audit trail completeness and immutability
- Trade reporting accuracy

5.6 Regression Testing

- Automated regression suite for critical order flows
- Executed for every sprint release

5.7 User Acceptance Testing (UAT)

- Business validation of trading workflows
- Real-world trading scenarios

---

6. Test Environment Strategy

Environments

- DEV – Developer testing
- QA – Functional & integration testing
- UAT – Business validation
- PROD – Live trading
- (Optional) DR – Disaster recovery validation

Environment Requirements

- Broker simulation or certification environment
- Production-like data setup (masked for PII)
- Stable integration endpoints

---

7. Test Data Management

- Use of **synthetic and masked production data**

- Coverage for:
    - Valid/invalid orders
    - Edge cases (large volumes, boundary values)
- Secure handling of **PII data**
- Data refresh strategy for test environments

---

8. Automation Strategy

- Automation integrated with **Azure DevOps pipelines**
- Focus areas:
    - Regression testing
    - API/FIX message validation
- Tools (indicative):
    - API testing tools (e.g., REST clients, FIX simulators)
- Trigger:
    - On every build and release cycle

---

9. Entry & Exit Criteria

Entry Criteria

- Requirements finalized and approved
- Test environment ready
- Test data prepared
- Build deployed successfully

Exit Criteria

- All critical and high defects resolved
- टेस्ट cases executed with ≥95% pass rate
- No open Sev-1/Sev-2 defects
- UAT sign-off received
- Compliance validation completed

---

10. Defect Management

- Defect tracking via **Azure DevOps**
- Severity classification:
    - Sev-1: Critical (system down, incorrect trades)
    - Sev-2: High (major functionality impacted)
    - Sev-3: Medium
    - Sev-4: Low
- Daily triage meetings in sprint cycles

---

11. Risks & Mitigation

Risk	Impact	Mitigation
Latency issues in order processing	Trade delays, financial loss	Performance testing, monitoring, optimization
Broker integration failures	Order routing failures	Mock testing, retry validation
Regulatory non-compliance (FINRA)	Legal penalties	Compliance-focused test cases
PII data exposure	Security breach	Data masking, encryption validation
Incomplete audit trails	Audit failure	Audit validation testing

---

12. Governance & Reporting

- Daily status tracking during sprint
- Sprint QA reports (execution, defects, coverage)
- Test metrics:
    - Test execution progress
    - Defect density
    - Pass/fail rate
- Stakeholder reporting:
    - QA Lead → Product Owner → Business stakeholders

---

13. Deliverables

- Test Strategy Document
- Test Plan (Sprint-level)
- Test Cases & Test Data
- Automation Scripts
- Test Execution Reports
- Defect Reports
- UAT Sign-off
- Audit Validation Reports

---

14. Tools & Technology

- Test Management & Defects: **Azure DevOps**
- Integration Testing: FIX simulators / API tools
- Automation: CI/CD pipelines (Azure DevOps)

---

15. Compliance & Audit

- Ensure full adherence to **FINRA regulations**
- Maintain:
    - Complete audit trails for all order activities

    - Timestamp accuracy
    - Data retention policies
- नियमित audit report validation as part of test cycles

---

16. Conclusion

This strategy ensures a **high-quality, compliant, and performant OMS system** through:

- Risk-based and agile testing
- Strong focus on **latency, compliance, and data security**
- Continuous validation via automation and CI/CD

Task2: Create a automation framework structure for mobile app testing using Appium and Perfecto Cloud using a chain-of-thought approach
Example Taken : Mobile App automation Appium + Perfecto cloud

Prompt :
Context : As a SDET create a well structured automation framework for mobile application testing using Appium integrated with Perfecto Cloud.

Instruction :
Ask me 5 question for Automation framework structure
After providing the answers ask the next question
Ensure the framework includes BDD Integration
Support for parallel execution in Perfecto cloud
Constraints : Follow the Industry coding standard

Output:

Question 1 of 5

👉 What **mobile platforms and application types** do you need to support?

Please specify:

- Platforms: (Android, iOS, or both)
- App type: (Native, Hybrid, Web)
- Device strategy: (Real devices on Perfecto, simulators/emulators, or both)

This will determine the **driver factory design, capabilities setup, and parallel execution strategy**.

Once you answer, I'll move to the next question.

both,Hybrid,Real devices on Perfecto

Great—that's clear: **Android + iOS**, **Hybrid apps**, and **real devices on Perfecto Cloud**. This will drive a **capability-driven, cross-platform design with WebView handling**.

Question 2 of 5

👉 What **programming language and test stack** do you prefer for the framework?

Please specify:

- Language: (Java, JavaScript/TypeScript, Python, etc.)
- BDD tool: (e.g., Cucumber, Behave, SpecFlow)
- Build tool: (e.g., Maven, Gradle, npm)
- Test runner: (e.g., TestNG, JUnit, Mocha)

This will define the **project structure, dependency management, and BDD integration layer**.

Once you answer, I'll move to the next question.

Java,Cucumber,Maven,TestNG

Perfect—this gives us a strong **enterprise Java + Cucumber + TestNG BDD foundation with Maven build management**.

We now move toward execution architecture and scaling.

Question 3 of 5

👉 What is your required **execution strategy for parallel runs in Perfecto Cloud?**

Please specify:

- Level of parallelism (e.g., number of devices/sessions at once or "unlimited via grid")
- Execution model: (feature-level parallel, scenario-level parallel, or both)
- Triggering method: (CI/CD via Jenkins, Azure DevOps, local runs, or mixed)
- Any tagging strategy required (e.g., smoke, regression, sanity, release-based tags)

This will define the **TestNG suite design, Cucumber parallel runner strategy, and Perfecto grid utilization model**.

Once you answer, I'll proceed to Question 4.

unlimited via grid,feature-level parallel,Azure DevOps,regression

Excellent—this clearly establishes a **cloud-scaled, CI-driven regression framework with feature-level parallel execution on Perfecto Grid via Azure DevOps**.

Now we'll define framework structure depth and engineering standards.

Question 4 of 5

👉 What **framework architecture standards and design patterns** do you want to enforce?

Please specify preferences for:

- Design pattern: (Page Object Model, Screenplay, Hybrid, etc.)
- Layering structure: (e.g., UI layer, business layer, utility layer, step definitions separation)
- Reporting requirements: (Allure, ExtentReports, custom HTML, Perfecto reporting integration)
- Logging standard: (Log4j, SLF4J, etc.)
- Test data strategy: (static data, external files, API-driven, or dynamic generation)

This will define the **core framework design, maintainability, and enterprise coding standards alignment**.

Once you answer, I'll move to the final question and then generate the complete framework blueprint.

Page Object Model,step definitions separation,Perfecto reporting,Log4j,API-driven

Great—this is now shaping into a **production-grade, scalable Appium + Perfecto + BDD automation framework** with strong enterprise patterns.

We're at the final design input.

Question 5 of 5

👉 What is your required **test coverage strategy and CI/CD quality gates**?

Please specify:

- Coverage focus: (Smoke, Regression, End-to-End, or all)
- Build quality gates in Azure DevOps:
  - Minimum pass % threshold?
  - Mandatory test suites (e.g., smoke must pass before deployment?)
- Failure handling strategy:
  - Retry mechanism? (yes/no)
  - Flaky test handling approach
- Any environment strategy:
  - DEV / QA / UAT usage for automation runs?

Once you answer, I will generate your **complete, enterprise-standard automation framework design document** including:

- Architecture
- BDD structure
- Perfecto parallel execution model
- Folder structure
- CI/CD pipeline design
- Coding standards
- Reporting strategy
- Best practices (industry aligned)

Regression,QA

Great—final inputs confirmed:

- Coverage: **Regression**
- Execution environment: **QA**

I'll now generate your **enterprise-grade automation framework design document**.

AUTOMATION FRAMEWORK DESIGN DOCUMENT

Mobile Test Automation Framework (Appium + Perfecto Cloud + BDD)

---

1. Objective

Design a **scalable, maintainable, and cloud-ready mobile automation framework** for **Hybrid mobile applications (Android & iOS)** using:

- **Appium**
- **Perfecto Cloud (Real Devices)**
- **BDD (Cucumber)**
- **TestNG**
- **Java + Maven**

The framework ensures:

- High scalability via **parallel execution in Perfecto Grid**
- Strong maintainability via **Page Object Model**
- Business-readable test scenarios via **BDD**
- Seamless CI/CD integration via **Azure DevOps**

---

2. High-Level Architecture

2.1 Framework Layers

Presentation Layer

- Cucumber Feature Files (Gherkin syntax)
- Business-readable test scenarios

Step Definition Layer

- Maps Gherkin steps → Java methods
- No business logic

Page Object Layer (POM)

- UI element locators
- Page-specific actions
- Encapsulation of mobile screens

Business Logic Layer (Service Layer - optional enhancement)

- Complex workflows across multiple pages

Core Framework Layer

- Driver Factory (Appium + Perfecto integration)
- Capability Manager
- Config Reader
- Logging (Log4j)
- Wait utilities
- Retry mechanism

Test Execution Layer

- TestNG Runner
- Parallel execution controller

---

3. Technology Stack

- Language: **Java**
- Automation: **Appium**
- Cloud Device Lab: **Perfecto Cloud**
- BDD: **Cucumber**
- Test Runner: **TestNG**
- Build Tool: **Maven**
- Logging: **Log4j**
- Reporting: **Perfecto Reporting Integration**
- CI/CD: **Azure DevOps**

---

4. Project Structure (Industry Standard)

```
src
 ├── main
 │    ├── java
 │    │    ├── base
 │    │    │    ├── BaseTest.java
 │    │    │    ├── DriverFactory.java
 │    │    │    ├── CapabilitiesManager.java
 │    │    ├── pages
 │    │    │    ├── LoginPage.java
 │    │    │    ├── HomePage.java
 │    │    ├── steps
 │    │    │    ├── LoginSteps.java
 │    │    ├── utils
 │    │    │    ├── ConfigReader.java
 │    │    │    ├── WaitUtils.java
 │    │    │    ├── APIDataProvider.java
 │    │    │    ├── LoggerUtil.java
 │    │    ├── runner
 │    │    │    ├── TestRunner.java
 │
```

```
├── test
│   ├── resources
│   │   ├── features
│   │   │   ├── login.feature
│   │   │
│   │   ├── config.properties
│   │   ├── log4j.properties
│   │   ├── testdata.json
```

---

5. BDD Integration Strategy

5.1 Feature File Example

```
Feature: Login Validation
Scenario: Valid user login
  Given user launches application
  When user enters valid credentials
  Then user should land on home screen
```

5.2 Step Definition Mapping

- One-to-one mapping with feature steps
- No business logic in step definitions

---

6. Page Object Model Strategy

Principles

- One page = one class
- Locators private
- Actions exposed via methods
- No assertions inside page classes

Example

```
public class LoginPage {
    @FindBy(id = "username")
    private MobileElement username;
    public void enterUsername(String user) {
        username.sendKeys(user);
    }
}
```

---

7. Perfecto Cloud Integration Strategy

Key Design

- Driver instantiated using **RemoteWebDriver (Perfecto URL)**
- Capability injection via JSON config
- Device selection handled dynamically

Capabilities Example

```
{
  "platformName": "Android",
  "browserName": "",
  "deviceName": "Samsung.*",
  "automationName": "Appium"
}
```

Execution Model

- Cloud-based execution via Perfecto Grid
- No local device dependency
- Session-based execution per test

---

8. Parallel Execution Strategy (Critical Design)

Execution Model

- Feature-level parallel execution using TestNG
- Unlimited scaling via Perfecto Grid

Configuration

- `testng.xml` configured for parallel = "tests"
- Each feature runs in separate thread

Behavior

- Multiple devices triggered simultaneously
- Independent Appium sessions per test
- No shared state across tests

---

9. CI/CD Strategy (Azure DevOps)

Pipeline Stages

1. Build (Maven compile)
2. Unit checks (optional)
3. Regression execution trigger
4. Parallel execution in Perfecto
5. Report generation
6. Publish results

Trigger

- On every QA deployment
- Scheduled regression runs

---

10. Logging Strategy

- Framework: **Log4j**
- Log Levels:
  - INFO → test flow
  - DEBUG → API/data details
  - ERROR → failures

---

11. Reporting Strategy

Primary Reporting

- Perfecto execution reports (session-level)

Supplementary Reporting

- Cucumber HTML report
- TestNG default reports

Logging Correlation

- Each test linked to Perfecto session ID

---

12. Test Data Strategy

- **API-driven test data generation**
- Dynamic data creation during runtime
- No hardcoded test data
- External APIs or service stubs used

---

13. Test Execution Strategy

Scope

- Regression suite only

Environment

- QA environment exclusively

Execution Flow

- Azure DevOps triggers pipeline

- TestNG distributes feature execution
- Appium connects to Perfecto devices
- Parallel execution across grid

---

14. Quality Gates

- Regression suite must pass before QA sign-off
- Retry mechanism for flaky tests (controlled at framework level)
- Failure threshold enforcement via CI pipeline

---

15. Error Handling & Stability

- Retry failed tests (configurable)
- Explicit waits (no static sleeps)
- Device session recovery via Perfecto reconnect
- Screenshot capture on failure

---

16. Coding Standards

- Clean Code principles (SOLID)
- No logic in Step Definitions
- No assertions in Page Objects
- Reusable utilities only in helper layer
- Consistent naming conventions:
    - Pages → `*Page`
    - Steps → `*Steps`
    - Utils → `*Util`

---

17. Key Benefits of This Framework

- Fully **cloud-scalable execution**
- Strong **BDD readability for business users**
- Highly maintainable via **POM architecture**
- Enterprise-grade **CI/CD automation**
- Optimized for **parallel regression execution**
- Minimal flakiness via API-driven data + retry logic